

TEST PLAN

Restaurant App - Team 3, RUN 26

RELATED ARTIFACTS

Ref.	Name
https://git.epam.com/epm-edai/project-runs/run-26/team-3/serverless/restaurant-app	GitLab Repository
https://kb.epam.com/spaces/EPMEDAI/pages/2812087677/User+Stories+R26	User Stories
https://run26-tm3-project-education-artifacts-dev.s3.eu-west-3.amazonaws.com/subin/r	Test Coverage

CONTENTS

1. INTRODUCTION
2. SCOPE OF WORK
 - 2.1 Components and Functions to be Tested
 - 2.2 Components and Functions Not to be Tested
 - 2.3 Third-Party Components
3. QUALITY AND ACCEPTANCE CRITERIA
4. CRITICAL SUCCESS FACTORS
5. RISK ASSESSMENT
6. RESOURCES
 - 6.1 Key Project Resources
 - 6.2 Test Team
 - 6.3 Test Environment
 - 6.3.1 Test Tools
7. TEST STRATEGY
 - 7.1 Entry Criteria
 - 7.2 Test Methods
 - 7.3 Test Types
 - 7.4 Test Levels
 - 7.4.1 Smoke Test
 - 7.4.2 Critical Path Test
 - 7.4.3 SQS Trigger E2E Test [NEW]
 - 7.4.4 Full API E2E Test [NEW]
 - 7.4.5 Extended Test
 - 7.5 Bug and Documentation Tracking
 - 7.5.1 Bug Severity Definitions

1 INTRODUCTION

This document describes the approach and methodologies used by the testing team to plan, organise, and perform the testing of the Restaurant App backend application for Team 3, RUN 26.

The Restaurant App consists of a RESTful backend API and associated services hosted on AWS. The backend is implemented as a serverless application using AWS Lambda and API Gateway, with AWS DynamoDB as the primary data store and AWS Cognito for authentication and user management. The application is designed to provide an online restaurant reservation platform, enabling customers to browse restaurant locations, view available tables, make reservations, explore the menu, and leave feedback. Waiters and administrators have additional capabilities for managing reservations, placing orders, and accessing performance reports.

This revision of the test plan extends the original RUN 26 document to cover the additional features delivered during the sprint: order management (POST /orders), customer feedback creation and editing (POST/PUT /feedbacks), admin reporting (GET /reports), SQS-driven report recalculation, and the fully automated end-to-end test suite in automation-qa/.

2 SCOPE OF WORK

2.1 COMPONENTS AND FUNCTIONS TO BE TESTED

#	Application / Component Name	Function Name
1	Registration Endpoint POST /auth/	New user registration with email and password. Validate input fields (email format, password policy). Automatic
2	Login Endpoint POST /auth/sign-in	Existing user login. Return access + refresh tokens. Account lockout after 5 consecutive failures (15-minute cool
3	Token Refresh POST /auth/refresh	Accept a valid refresh token. Return a new access token. Reject expired or invalid tokens.
4	Logout POST /auth/logout	Revoke the provided refresh token. Subsequent requests with revoked token are rejected.
5	Role Management	Customer by default; Waiter if email in waiter-emails whitelist; Admin if email in admin-emails whitelist.
6	User Profile GET/PUT /users/profile	Return and update authenticated user profile data. Validate updated fields. Reject unauthenticated requests.
7	Locations GET /locations, /location	Return list of locations and single-location details. Return 404 for non-existent IDs.
8	Speciality & Popular Dishes	GET /locations/{id}/speciality-dishes and GET /dishes/popular. Return correct dish lists per location.
9	Feedbacks GET /locations/{id}/feed	Paginated feedback list. Filter by type (Cuisine / Service). Sort by date or rating. Correct pagination metadata.
10	Available Tables GET /bookings/ta	Tables filtered by location_id, date, guest_count, optional time window. Each result shows 90-minute slot availa
11	Create Reservation POST /bookin	Authenticated customer creates a reservation. Deterministic waiter assignment. Reservation created with RESE
12	List / Detail Reservations GET /boo	List and fetch individual reservations. Customer sees own; waiter sees assigned location.
13	Update Reservation PUT /booking	Allow status transitions per role. Validate allowed transitions.
14	Cancel Reservation DELETE /boo	Customer cancels a RESERVED reservation. Status transitions to CANCELLED.
15	Waiter Reservation View GET /res	Waiter table-view filtered by date, time, and table name. Role guard: customer ? 403.
16	Order Management POST /orders	Waiter places food/drink order for an in-progress reservation. Items validated (dishId, quantity ? 1). Only the ass
17	Feedback Submission POST /feed	Customer submits CULINARY or SERVICE feedback after a FINISHED reservation. Rating 1-5. Deterministic U
18	Feedback Edit PUT /feedbacks [N	Customer edits their own feedback (rating + comment). SQS SERVICE EDITED event recalculates waiter-repor
19	Feedback Context GET /feedbacks	Returns waiter_id and feedback metadata for the feedback modal. Customer-only; waiter ? 403.
20	Admin Reports GET /reports [NEV	Admin retrieves staff_performance or sales report for a given period. Optional locationId filter. Role guard: non-a
21	SQS Report Recalculation [NEW]	FINISHED reservation event recalculates waiter-report (orders_processed, avg_service_feedback) and location
22	Input Validation and Error Handling	All endpoints return structured JSON error responses. 400 bad request, 401 unauthorized, 403 forbidden, 404 n

Rows highlighted in green are new items added in this revision.

2.2 COMPONENTS AND FUNCTIONS NOT TO BE TESTED

The functionalities listed below are out of scope due to time constraints, sprint backlog priority, or not yet being implemented.

#	Application / Component Name	Function Name
---	------------------------------	---------------

1	Anonymous Feedback	Non-registered visitor feedback without an account
2	Report Export	Exporting reports in PDF, Excel, or CSV formats
3	Customer Pre-order	Pre-orders placed at reservation time forwarded to kitchen
4	User Password Recovery	Forgot-password email flow and reset via token
5	CAPTCHA during Registration	Bot-prevention CAPTCHA on sign-up
6	Admin Dashboard UI	Administrator management interface (UI layer)
7	Social Media Login	OAuth via Google, Facebook, or other providers
8	AI Feedback Moderation	Automated AI moderation of customer reviews
9	AI Activity Analysis	AI-generated insights on reservation and revenue patterns
10	Frontend / UI Layer	Web application UI, cross-browser, responsive design

2.3 THIRD-PARTY COMPONENTS

#	Component Name	Component Role	Reference / Comment
1	AWS Cognito	User pool for authentication; issues JWT tokens	Required for all authenticated endpoints
2	AWS DynamoDB	Primary NoSQL database for all application data	14 tables with GSI support
3	AWS API Gateway	HTTP routing and Lambda integration	RESTful interface to the backend
4	AWS Lambda	Serverless compute for the API handler	Single Lambda function (ApiHandler)
5	AWS SQS	Event queue for reservation and feedback state	Drives async report recalculation

3 QUALITY AND ACCEPTANCE CRITERIA

- The product must work according to the requirements and functional specification listed in Scope of Work.
- The product bug level must reach acceptance criteria: no Blocker or Critical bugs at release. Major bugs must have documented workarounds or be scheduled for immediate hotfix.
- All API endpoints must return correct HTTP status codes and structured JSON responses.
- All authentication flows must correctly enforce role-based access control.
- Unit test code coverage must be at least 80% of application source lines.
- All automated tests (pytest unit + E2E suite) must pass in the CI pipeline (GitLab CI).
- API response times under normal load must not exceed 3 seconds for any endpoint.
- SQS-driven report recalculation must complete within 15 seconds of the triggering event under normal AWS load.

4 CRITICAL SUCCESS FACTORS

- Complete development and testing of all Sprint 1 functionalities within the sprint timeline.
- Application must not have known Blocker or Critical bugs at the time of the sprint demo.
- Functional requirements must not have last-minute changes without updated test coverage.
- Automated unit tests cover all core business logic paths (registration, login, lockout, reservation, orders, feedback, reports, SQS recalculation).
- All team members have access to the test environment and GitLab repository.
- The E2E automation suite (test_e2e_endpoints.py) passes in a clean seeded environment.

5 RISK ASSESSMENT

#	Risk	Probability	Status	Impact	Preventive Actions	Contingency Plan
1	Limited testing experience with AWS	High	Open	Slow test development	Mock examples + pair-programming	Focus on unit tests with mocked AWS
2	AWS Cognito configuration issues	Medium	Open	Auth endpoints unavailable	Set up local mock Cognito endpoints	Use mock Cognito in unit tests; flag in
3	API contract changes during sprint	Medium	Open	Test cases require rework	Daily sync; OpenAPI spec as source of truth	Prioritise stable endpoints; update inc
4	DynamoDB schema changes	Medium	Open	Test failures; incorrect data	Freeze schema early; use seed data	Update repositories and tests together
5	SQS event delivery delay causing timeouts	Medium	Open	False failures in T-4	Poll DynamoDB for up to 15 seconds	Retry failed assertions once; log diagn
6	Insufficient time to cover all 20+ endpoints	Low	Open	Reduced coverage	Prioritise smoke and critical-path tests	Defer extended tests; document untes
7	Team member unavailability during sprint	Low	Open	Reduced testing capacity	Cross-train team members on test env	Redistribute testing tasks; reduce sco

6 RESOURCES

6.1 KEY PROJECT RESOURCES

#	Project Role	Name / Email / Location
1	Developer	Mateja Subin
2	Developer	Emilija Dotlic
3	Developer	Kristijan Kovacevic
4	Developer	Tadej Dragojlovic

6.2 TEST TEAM

#	Project Role	Name	Location	Responsibilities
1	Developer / QA	Mateja Subin	Serbia	Creating test documentation, writing unit tests, test execution, bug reporting
2	Developer / QA	Emilija Dotlic	Serbia	Writing unit tests, test execution, requirements analysis
3	Developer / QA	Kristijan Kovacevic	Serbia	Writing unit tests, test execution
4	Developer / QA	Tadej Dragojlovic	Serbia	Writing unit tests, test execution

6.3 TEST ENVIRONMENT

- Backend Application: AWS Lambda + API Gateway (staging environment, eu-west-3)
- Authentication: AWS Cognito User Pool (staging)
- Database: AWS DynamoDB (staging tables with seed data, 14 tables)
- Message Queue: AWS SQS (reservation/feedback events driving report recalculation)
- Local Development: Python 3.14, pytest with mocked AWS services
- Devices: N/A (API-only backend; no UI testing in scope)
- Operating Systems: Windows 11, macOS (developer machines for local test runs)
- CI/CD: GitLab CI pipeline (git.epam.com/epm-edai/project-runs/run-26/team-3)

6.3.1 Test Tools

#	Tool	Comment
1	Microsoft Teams	Communication and documentation-sharing channel
2	GitLab	Version control. All test scripts stored in project repository
3	pytest v9.0.3	Primary unit test framework. Executes all backend unit tests
4	pytest-cov	Code coverage reporting after test runs
5	unittest.mock	Mock AWS services (DynamoDB, Cognito, SQS) in unit tests
6	Postman	Manual API testing: request/response validation, exploratory testing
7	ruff	Code linting via pre-commit hooks
8	test_sqs_endpoints.py	Automated SQS trigger E2E test (T-1 - T-12) with DynamoDB polling
9	test_e2e_endpoints.py	Full API E2E suite: 11 suites, 114 steps, PDF + JSON output
10	fpdf2	PDF generation for test_output.pdf and this test plan

Rows highlighted in green are new tools added in this revision.

7 TEST STRATEGY

The Team 3 Restaurant App backend is tested using a "black box" approach based on the API requirements and functional specification, without direct knowledge of internal implementation details. Testing focuses on the API contract: correct HTTP methods, paths, request bodies, response schemas, status codes, and business rule enforcement.

7.1 ENTRY CRITERIA

Testing may be suspended if any of the following occurs:

- It is impossible to deploy the application to the test environment following the README.
- A fault with a feature prevents its testing (e.g., Lambda function not responding).
- Application does not contain the specified change(s) from the sprint backlog.
- A core endpoint (authentication, reservation creation) is entirely non-functional.
- A severe environment problem (DynamoDB unavailable, Cognito misconfigured) that prevents testing from continuing.
- Development has not corrected a previously identified blocking issue.

7.2 TEST METHODS

Testing is the process of verifying that the Restaurant App backend operates in compliance with its functional specification and API contract.

- Manual functional testing - Postman for exploratory and ad hoc verification of individual endpoints, especially for edge cases and error scenarios.
- Automated unit testing - Python pytest with mocked AWS services. All business logic services and repositories are covered. Primary method for Sprint 1.
- SQS trigger E2E testing - test_sqs_endpoints.py drives a full reservation lifecycle and polls DynamoDB waiter_report and location_report tables after each event, asserting before/after diffs.
- Full API E2E automation - test_e2e_endpoints.py seeds a clean environment, executes 11 suites (114 steps) covering every route group, verifies HTTP responses and DynamoDB state, and writes a PDF report.

7.3 TEST TYPES

Functional Testing

Validates that the system operates in compliance with the defined requirements. Covers user registration, authentication, role assignment, reservation management, location/menu retrieval, order placement, feedback, and reporting.

Regression Testing

Confirms that bug fixes and new features do not break existing functionality. Executed automatically via pytest on every build.

Integration Testing

Verifies that the Lambda handler correctly routes requests, that repositories correctly interact with DynamoDB, and that Cognito token flows integrate with protected endpoints.

Event-Driven Testing [NEW]

Validates that SQS events (FINISHED, CANCELLED, CREATED, EDITED) are processed correctly by WaiterReportService and LocationReportService. DynamoDB report tables are polled after each event to assert expected changes.

API Contract Testing

Ensures all endpoints return correct HTTP status codes, response body structures, and headers. Covers both success paths and error paths (400, 401, 403, 404, 409, 422).

Security Testing

Validates that protected endpoints reject unauthenticated requests, that role-based access control is enforced, and that JWT tokens are correctly validated.

Smoke Testing

High-level verification that the core system is functional: login, list locations, create a reservation, place an order, submit feedback. Acts as a deployment gate before deeper testing.

Ad Hoc Testing

Informal, unscripted exploration of the API to uncover unexpected behaviour or edge cases not covered by formal test cases.

7.4 TEST LEVELS

7.4.1 Smoke Test

Performed after each deployment. Covers the most critical paths to assess build stability before deeper testing.

1. POST /auth/sign-up - register a new customer account
2. POST /auth/sign-in - sign in with the registered account
3. GET /locations - retrieve the list of restaurant locations
4. GET /bookings/tables - retrieve available tables for a location
5. POST /bookings/client - create a reservation
6. GET /bookings/client - retrieve the user's reservations
7. PUT /bookings/waiter/{id} - waiter moves reservation to In Progress
8. POST /orders - waiter places an order [NEW]
9. PUT /bookings/waiter/{id} - waiter marks reservation Finished [NEW]
10. POST /feedbacks - customer submits culinary feedback [NEW]
11. DELETE /bookings/client/{id}/cancel - cancel a reserved reservation
12. POST /auth/logout - log out

If Smoke Test fails, the Testing Team notifies via Teams and suspends further testing until a corrected build is available.

7.4.2 Critical Path Test

Performed after Smoke Test passes. Covers all sprint user stories in full, including validation edge cases, role-based access control, lockout behaviour, pagination, sorting, and multi-slot reservation logic. Executed using the full pytest suite (35 test modules). All modules must pass.

7.4.3 SQS Trigger E2E Test [NEW]

Validates that SQS events emitted by reservation and feedback state changes correctly recalculate the waiter_report and location_report DynamoDB tables. Test script: backend/test_sqs_endpoints.py. Uses Airport location and max@example.com (zero-baseline) so every change is visible.

Step	Scenario	SQS Event	Expected Change
T-1	kate creates reservation at Airport table 1	CREATED	reports UNCHANGED
T-2	max moves RESERVED ? IN_PROGRESS	UPDATED	reports UNCHANGED
T-3	max creates order (2× Grilled Chicken Wrap)	none	-
T-4	max marks reservation FINISHED	FINISHED	orders_processed +1; revenue increases
T-5	kate submits CULINARY feedback (rating 5)	CUL. CREATED	cuisine_count +1; avg ? 5.0
T-6	kate submits SERVICE feedback (rating 4)	SVC. CREATED	service_count +1; avg ? 4.0
T-7	duplicate CULINARY feedback ? 409	none	location-report UNCHANGED
T-8	duplicate SERVICE feedback ? 409	none	waiter-report UNCHANGED
T-9	kate creates 2nd reservation (slot B)	CREATED	reports UNCHANGED
T-10	kate cancels 2nd reservation	CANCELLED	reports UNCHANGED
T-11	kate edits SERVICE feedback 4?1 [NEW]	SVC. EDITED	avg_service_feedback DROPS; count un
T-12	kate edits CULINARY feedback 5?1 [NEW]	CUL. EDITED	avg_cuisine_feedback DROPS; count un

Rows highlighted in green are new test cases added in this revision.

7.4.4 Full API E2E Test [NEW]

Fully automated end-to-end validation of every API route group. Script: automation-qa/test_e2e_endpoints.py. Seeds a clean deployed environment via quick_seed.py, resolves API Gateway URL and DynamoDB table names at runtime, runs all suites in dependency order, and writes test_output.pdf + a JSON results file.

Suite	Steps	Routes Covered	DB Verification
AUTH	13	POST /auth/sign-up, /sign-in, /refresh, /logout	customers table row created; duplicate count stays at 1

LOCATIONS	15	GET /locations, /locations/{id}, /speciality-dishes	API count vs. DynamoDB table count; totalElements vs. feedback ro
DISHES	8	GET /dishes, /dishes/popular, /dishes/{id}	-
USERS	8	GET/PUT /users/profile, GET /users/waiter/locat	fname updated and restored in customers table
CUSTOMERS	3	GET /customers	API count vs. customers table count
REPORTS	9	GET /reports (staff_performance, sales)	-
BOOKINGS	18	GET/POST/PUT /bookings/*, DELETE /booking	reservation row status and guest count; slot status in slots table
ORDERS	8	POST /orders	orders row created with correct items
WAITER RESERVATIONS	5	GET /reservations/waiter	-
BOOKINGS LIFECYCLE	6	PUT /bookings/waiter/{id} finish, POST slot B, D	status transitions verified in reservations table
FEEDBACKS	14	GET/POST/PUT /feedbacks, GET /feedbacks/d	feedback_cuisine and feedback_service rows; rate updates

7.4.5 Extended Test

Covers less critical scenarios: pagination boundaries, concurrent booking conflicts, turnover gap edge cases, and invalid filter combinations. Scheduled later in the sprint if time permits.

7.5 BUG AND DOCUMENTATION TRACKING

GitLab Issues are used for bug reporting and documentation tracking. Each bug report must include: title, steps to reproduce, expected result, actual result, severity label, and relevant logs or screenshots. Bug metrics are summarised in the Sprint Test Result Report.

7.5.1 Bug Severity Definitions

1. Blocker

The application or a critical feature is entirely non-functional; no workaround. Example: Lambda crashes on any request; sign-in always returns 500.

2. Critical

Severe issues that break main functionalities; no practical workaround. Example: Reservation creation silently succeeds but no DynamoDB record is created; JWT tokens not validated on protected endpoints.

3. Major

Significant defects affecting important (non-critical) functionality; workarounds exist but are inconvenient. Example: Account lockout not resetting after 15 min; SQS event not triggering report recalculation.

4. Minor

Issues that cause inconvenience but do not hinder critical operations. Example: Incorrect field name in error response; average rating off by rounding.

5. Trivial

Cosmetic or negligible issues with no impact on functionality. Example: Inconsistent casing in response field names; extra whitespace in error messages.